

标定视觉基础模型的泛化边界：面向未见物体位姿估计的自适应层选择

技术可行性报告 · 2026-07-21 · idea: 自适应基础模型层选择_标定DINOv2在未见物体位姿估计中的泛化边界.md · ReAct 写作（边写边查证 papers/cards/codebases） · model: qwen3.8-max-preview 技术可行性报告 · 2026-07-21 · idea: 自适应基础模型层选择_标定DINOv2在未见物体位姿估计中的泛化边界.md 依据：领域精读卡片库（cards/）· 领域母题（_themes.json）· repo 卡（codebases/foundpose.md）· 查重与对抗评审记录

1. 背景与动机

1.1 问题陈述

未见物体（unseen object）6D位姿估计要求系统在推理阶段对训练集中从未出现的物体直接输出精确的旋转矩阵 $R \in SO(3)$ 和平移向量 $\mathbf{t} \in \mathbb{R}^3$ ，仅依赖物体的3D网格模型或有限参考视图。自DINOv2被FoundPose（*Unseen Object Pose Estimation with Foundation Features*）引入该任务以来，视觉基础模型（VFM）的patch描述子已成为事实上的共享特征基础设施：FoundPose论文使用DINOv2 ViT-L/14的第18层token特征做模板检索与PnP求解（papers/foundpose.md第293行），而其开源repo实际配置为ViT-S/14的第9层（codebases/foundpose.md F3: configs/infer/lmo.json:12 与 configs/gen_repre/lmo.json:7 均写定 layer=9）；GS-Pose（GS-Pose: Generalizable Segmentation-based 6D Object Pose Estimation with 3D Gaussian Splatting）用DINOv2通用特征驱动跨物体分割；Cross-View方法（Learning Cross-View Semantic Priors for Single-Reference Unseen Object Pose Estimation）在VFM密集token层上建立跨视图语义先验，其默认骨干为DINOv3预训练 ViT-Base（cards/learning_cross_view.json resources字段）。

当前瓶颈的核心表征：上述路线均依赖视觉基础模型，但在层选择上无统一依据——FoundPose论文描述使用ViT-L/14（24层）的第18层，而repo实际使用ViT-S/14（12层）的第9层，二者均为无理论依据的单点固定选择。DINOv2 ViT-L/14拥有24个block（codebases/foundpose.md F10: layer=23 为最后层），ViT-S/14拥有12个block（block 0 – 11），不同深度的block在语义抽象层次、位置敏感性和纹理响应上存在系统性差异，而现有工作对此完全忽视。

具体而言，可量化的瓶颈表现在以下三个维度：

物体属性	典型数据集	已知失效模式	根源
低纹理（工业零件等）	BOP T-LESS	梯度稀疏，patch描述子近似常数向量	浅层纹理特征主导但无梯度；深层语义过度平滑
旋转对称	BOP T-LESS、LM-O（eggbox/glue）	歧义位姿均匀分布，单层特征无法区分	对称轴方向token高度相似，匹配退化
反光/镜面	T-LESS（金属工业件）	高光区域特征方差爆炸，检索不稳定	浅层对高频纹理过于敏感

BOP基准报告（*BOP: Benchmark for 6D Object Pose Estimation*）显示，LM与LM-O之间的召回率差距超过30个百分点（papers/bop.md第181 – 182行），遮挡和特殊材质是首要因素。领域母题第2条evidence字段中BundleTrack指出“对无纹理、反光、扁平物体跟踪仍具挑战性”（cards/_themes.json）；综述（Deep Learning-Based Object

Pose Estimation: A Comprehensive Survey, 2025) 亦将“缺乏纹理/显著形状特征”列为挑战性场景下鲁棒性不足的表现 (cards/deep_learning_based.json limitation字段)。固定层选择在上述场景下的退化是系统性的, 而非偶发性的。ZS6D (ZS6D: Zero-shot 6D Object Pose Estimation using Vision Transformers) 提供了直接实证: 该方法以DINOv2 ViT-S/8描述子做零样本位姿估计, 在T-LESS上使用CNOS掩码时AR仅为0.210, 远低于LMO的0.298和YCBV的0.324 (papers/zs6d.md Table II), 作者明确承认“TLESS上无纹理对称物体导致局部对应歧义” (cards/zs6d.json limitation字段)。这一失败并非孤例, 而是语义训练特征在无纹理/对称物体上丧失判别力的系统性表现。领域母题「视觉基础模型 (DINOv2/MASSt3R/SAM) 被默认为‘即插即用的通用几何-语义特征源’, 但其跨域、跨尺度、跨模态的泛化边界从未被系统界定」 (cards/_themes.json) 进一步将两个开放问题显式列出: (1)“选择哪一层 (第9层vs第18层) 完全靠经验, 缺乏理论指导”; (2)“无人提出特征可靠性的在线检测机制”。该母题同时揭示了一个更深层的张力: ‘冻结特征足够好’的隐含假设使整个领域回避了‘何时该微调、微调多少’的核心问题——本研究的自适应层选择+可靠性门控+MASSt3R回退正是对该问题的零重训练回答: 不微调, 但系统地界定冻结特征的适用边界并在越界时切换特征源。作为补充, 若标定结果揭示冻结特征在特定属性子集上系统性失效, LoRA轻量域适配 (如对ViT最后若干层施加低秩适配, 仅需每物体5-10张标注图) 可作为“完全冻结”与“全量微调”之间的中间方案, 以最小训练代价恢复工业场景判别力。本研究正是对上述开放问题的直接回应。

1.2 相关工作

1.2.1 基于CAD模型的渲染比较路线

该路线以MegaPose (MegaPose: 6D Pose Estimation of Novel Objects via Render and Compare) 为代表, 采用粗估计器 (模板分类) + 迭代精炼器 (渲染比较) 的两阶段架构, 在约200万合成图像上训练以弥合sim-to-real gap (cards/megapose.json method字段)。MegaPose精炼器推理单步需66.5ms (cards/megapose.json limitation字段), 且依赖高质量CAD模型。FoundPose在此路线上取消了神经网络训练, 改用DINOv2中间层描述子直接做TF-IDF模板检索, 最优性能仍需依附MegaPose精炼器 (cards/foundpose.json limitation字段), 削弱了“完全无需训练”的纯粹性。RayPose (RayPose: Ray Bundling Diffusion for Template Views in Unseen 6D Object Pose Estimation) 将模板匹配重构为射线束对齐的扩散问题, 以生成多假设缓解检索失败, 但精炼阶段同样依赖MegaPose refiner (cards/raypose.json limitation字段)。

1.2.2 基于参考视图的model-free路线

Gen6D (Gen6D: Generalizable Model-Free 6-DoF Object Pose Estimation from RGB Images) 用3D特征体积 (分辨率 32^3) 做model-free精炼 (cards/gen6d.json limitation字段), 其核心贡献是将参考图像2D特征反投影到3D体素空间; GS-Pose进一步以3D Gaussian Splatting替代隐式体积, 支持基于梯度的迭代位姿优化。两者均需已知位姿的多视角参考图像, 且GS-Pose离线构建3DGS模型的前提限制了动态场景应用 (cards/gs_pose.json limitation字段)。OnePose++ (OnePose++: Keypoint-Free One-Shot Object Pose Estimation without CAD Models) 去除关键点检测依赖, 但在对称物体上仍有明显性能差距——如glue的ADD(S) 仅48.0, 而PVNet达95.7 (cards/onepose*.json limitation字段)。

1.2.3 基于VFM的新兴路线

FoundPose、GS-Pose、Cross-View Semantic Priors三个方法共同依赖视觉基础模型 (FoundPose/GS-Pose用DINOv2, Cross-View默认用DINOv3 ViT-Base), 但假设方向截然不同: FoundPose假设中间层描述子具备跨合成-真实域泛化能力 (cards/foundpose.json core_assumption); GS-Pose假设DINOv2通用特征在无微调条件下足

以支持跨物体分割 (`cards/gc_pose.json core_assumption`) ; Cross-View假设VFM密集token已编码可用于跨视图判别的外观信息 (`cards/learning_cross_view.json core_assumption`) 。三者均未系统测试VFM特征的失效边界, 亦未针对物体属性自适应地调整层选择策略。PoseGAM (PoseGAM: Robust Unseen Object Pose Estimation via Geometry-Aware Multi-View Reasoning) 采用多视图基础模型架构, 指出CAD渲染与真实观测的外观差异对特征质量的影响 (`cards/posegam.json limitation`字段), 但仍未触及层选择问题。ZS6D (*ZS6D: Zero-shot 6D Object Pose Estimation using Vision Transformers*) 同样以DINOv2 ViT-S/8为特征骨干, 在T-LESS上因无纹理对称物体导致局部对应歧义而性能骤降 (AR 0.210 vs LMO 0.298, `papers/zs6d.md Table II`), 是语义训练特征失效的直接证据。值得注意的是, MAST3R (Grounding Image Matching in 3D with MAST3R, `arxiv:2406.09756`) 通过InfoNCE损失对真实3D对应点训练24维局部描述子 (`cards/grounding_image_matching_in_3d_with_mast3r.json method`字段), 其训练目标为几何对应而非语义判别, 提供了“几何训练特征 vs 语义训练特征”的天然对照——若DINOv2的层敏感性根源在于语义训练目标, 则MASt3R特征应表现出更弱的属性依赖性。需注明两项边界条件: (1) MAST3R/DUST3R架构将输入缩放至最大边518像素, 高分辨率场景精度骤降 (`arxiv:2507.14798`), 但BOP管线中物体裁剪图通常为 420×420 px (`papers/foundpose.md`第290行), 低于该限制, 故对照实验在BOP上有效且信息损失有限; 若将结论推广至高分辨率工业检测场景 (4K+), 需额外考虑MASt3R的分辨率瓶颈。(2) G-MASt3R-SfM (`arxiv:2606.22856`) 揭示MASt3R对非重叠图像对仍会输出错误对应, 但FoundPose流水线中模板与查询图来自同一物体的不同视角 (800模板/物体, 约 25° 间隔), 视角重叠由模板采样策略保证, 该风险在架构层面已被规避。

1.2.4 评测基准

BOP (*BOP: Benchmark for 6D Object Pose Estimation*) 提供统一的AR指标 (VSD/MSSD/MSPD均值), 是当前最权威的跨数据集评测框架, 其VSD误差函数仅在可见表面区域计算对齐误差, 可等价处理对称物体的歧义位姿 (`cards/bop*.json core_assumption`字段)。本研究采用BOP的LM-O (低纹理+遮挡) 和T-LESS (工业对称件) 两个数据集作为主要实验场。

1.3 根本性分析

1.3.1 固定层选择为何失效：信号-层深度的属性依赖

FoundPose的推理流程为：描述子提取 → 最近邻模板检索 → PnP求解。层选择直接决定第一步输出的描述子质量, 而描述子质量对物体属性的依赖是非均匀的：

低纹理物体 (如T-LESS工业零件)：浅层 (小 ℓ) token主要响应局部边缘和色彩梯度。当纹理密度趋近零时, 浅层描述子退化为近常数向量——patch间余弦相似度趋近1, 位置判别信息消失。深层token编码了更强的全局语义和几何结构, 在无纹理场景下相对保留更多位置信息, 但过深的层引入过度语义化 (semantic over-smoothing), 相邻patch特征高度相关, 形状判别能力反而下降。因此低纹理物体的最优层位于中间偏浅位置, 而非默认 `layer=9` 。

对称物体 (如LM-O中的eggbox、glue)：深层token倾向于提取物体的整体语义类别信息, 对对称轴方向的细粒度几何差异不敏感, 使位姿歧义加剧；中浅层保留了更多局部几何细节 (表面法向变化), 有助于区分不同旋转状态。BOP的VSD指标从评测端处理了对称歧义, 但特征层本身并未——匹配阶段仍可能选到对称等价但错误的模板。

反光物体：高光像素的出现位置随视角连续变化, 在DINOv2浅层 (响应高频纹理) 表现为特征方差的大幅跳变, 破坏跨视图匹配稳定性；中深层对高光的响应被GeLU和LayerNorm平滑, 特征方差趋于稳定。

综合以上分析，最优层选择应为物体属性的函数 $\ell^*(\mathbf{a})$ ，而固定 $\ell = 9$ (FoundPose repo实际配置) 是对该函数在属性空间上的单点近似，其误差随物体属性偏离训练分布而增大。

1.3.2 系统架构层面的不可修正性

上述失效并非模型能力上限所致，而是系统设计造成的盲区。FoundPose的 `gen_repre` 阶段离线构建物体表示（特征提取→PCA→KMeans→TF-IDF），`infer` 阶段在线匹配。一旦 `gen_repre` 固定了层号，整个物体表示就被锁定在单一特征空间中——即使推理时发现当前层对某物体不佳，也无法在不重建物体表示的情况下切换。`_register_hooks` (codebases/foundpose.md F5: `dinov2_utils.py:198-223`) 虽已支持 `List[int]` 多层输入，但 `extract_descriptors` 只接收单个 `layer: int` 并包成 `[layer]` 传入，`forward` 又只调用一次——多层能力被上层接口“卡住”。

这意味着：当前系统对层选择错误无自我修正能力。修正路径不是重新训练模型，而是在 `gen_repre` 之前根据物体属性确定最优层——这正是本方案的设计出发点。

2. 方法

本研究提出自适应基础模型层选择 (Adaptive Layer Selection, ALS) 框架，以零重训练的方式将FoundPose的固定层配置扩展为属性条件自适应层选择。框架分三个互补贡献：C1负责标定DINOv2各层在不同物体属性下的性能边界；C2基于标定结果设计轻量级属性分类器；C3将自适应层选择插入FoundPose流水线。

Contribution 1: DINOv2层-属性性能标定实验

设计动机

在提出自适应方案之前，必须首先回答：哪些层在哪些属性条件下最优？现有工作 (FoundPose repo用 `layer=9`、论文描述用ViT-L/14第18层) 均为单点选择，缺乏系统性扫描数据。C1通过受控实验填补这一空白，其输出作为C2训练数据和C3决策规则的唯一事实依据。

MASt3R对照子假设：若DINOv2层选择问题的严重性与训练目标（语义判别 vs 几何对应）因果相关，则以几何对应为训练目标的MASt3R局部描述子（24维，InfoNCE损失对真实3D对应点训练，cards/grounding_image_matching_in_3d_with_mast3r.json）在低纹理/对称物体子集上的性能退化幅度应显著小于DINOv2任何单层。C1因此增设实验组B：以MASt3R描述子替换DINOv2，在相同物体分组条件下执行相同流水线，记录AR分数，对比两者的“属性敏感性曲线”。若MASt3R在低纹理/对称组上退化显著更小，则证实DINOv2层敏感性的根源是语义训练目标而非ViT架构本身。分辨率边界条件：MASt3R将输入缩放至最大边518 px (arxiv:2507.14798)，BOP模板尺寸为420×420 px (papers/foundpose*.md第290行)，处于该限制之内，故实验组B在BOP上信息损失有限；但若推广至更高分辨率场景，需将分辨率纳入自适应决策维度。此外，G-MASt3R-SfM (arxiv:2606.22856) 指出MASt3R对非重叠图像会对输出错误对应——本实验中模板与查询均为同一物体的不同视角，视角重叠由采样策略保证，该风险不影响实验组B的有效性。

技术细节

实验设计矩阵：

维度	取值	说明
DINOv2层 ℓ	$\{0, 2, 4, 6, 8, 9, 10, 11\}$	ViT-S/14共12层 (block 0–11)；均匀覆盖+默认层邻域加密 (见§3.4.3)
数据集	LM-O, T-LESS	覆盖低纹理+遮挡、工业对称件
物体类型 c	纹理丰富/低纹理/对称	按C2属性分类器的三分类方案标注
指标	AR (VSD/MSSD/MSPD均值)	BOP标准指标

共 $8 \times 2 \times 3 = 48$ 个实验条件，每条件在对应数据集的物体子集上运行FoundPose完整流水线。

层扫描实现：FoundPose的层号通过 `model_name` 字符串中的 `layer=N` 字段控制（codebases/foundpose.md F2: `dinov2_utils.py:60–78` 解析逻辑）。扫描脚本仅需循环修改 `extractor_name` 字段，无需改动模型代码：

```

# scan_layers.py
import json, subprocess, itertools, pathlib

LAYERS = [0, 2, 4, 6, 8, 9, 10, 11] # ViT-S/14 layer indices (0-indexed)
DATASETS = ["lmo", "tless"]
BASE_CFG_REPRE = "configs/gen_repre/{ds}.json"
BASE_CFG_INFER = "configs/infer/{ds}.json"
RESULTS = []

def make_extractor_name(layer: int) -> str:
    return (
        f"dinov2_version=vits14-reg_stride=14"
        f"_facet=token_layer={layer}_logbin=0_norm=1"
    )

for layer, ds in itertools.product(LAYERS, DATASETS):
    extractor_name = make_extractor_name(layer)

    repre_cfg_path = pathlib.Path(BASE_CFG_REPRE.format(ds=ds))
    cfg = json.loads(repre_cfg_path.read_text())
    cfg["extractor_name"] = extractor_name
    tmp_repre = pathlib.Path(f"/tmp/repre_{ds}_layer{layer}.json")
    tmp_repre.write_text(json.dumps(cfg))

    infer_cfg_path = pathlib.Path(BASE_CFG_INFER.format(ds=ds))
    cfg2 = json.loads(infer_cfg_path.read_text())
    cfg2["extractor_name"] = extractor_name
    tmp_infer = pathlib.Path(f"/tmp/infer_{ds}_layer{layer}.json")
    tmp_infer.write_text(json.dumps(cfg2))

    subprocess.run(
        ["python", "scripts/gen_repre.py", "--opts-path", str(tmp_repre)],
        check=True
    )
    subprocess.run(
        ["python", "scripts/infer.py", "--opts-path", str(tmp_infer),
         "--output", f"/tmp/result_{ds}_layer{layer}.json"],
        capture_output=True, check=True
    )

    ar_data = json.loads(
        pathlib.Path(f"/tmp/result_{ds}_layer{layer}.json").read_text()
    )
    for obj_type, ar in ar_data["per_type"].items():
        RESULTS.append({
            "layer": layer, "dataset": ds,
            "object_type": obj_type, "AR": ar
        })

json.dump(RESULTS, open("result.json", "w"), indent=2)

```

关键修改点（repo卡定位）： - configs/gen_repre/lmo.json:7 : extractor_name → 替换 layer=9 为目标层号（codebases/foundpose.md F3） - configs/infer/lmo.json:12 : extractor_name → 同上（codebases/foundpose.md F3） - utils/dinov2_utils.py:60-78 : model_name 字符串解析逻辑（F2），

无需修改，已支持 `layer=N` - `utils/dinov2_utils.py:56` : `self.layer: int = 9` (F1) 为默认值，通过 `model_name` 解析覆盖，扫描脚本不触碰此处

输出格式: `result.json` 结构为 `{layer: int, dataset: str, object_type: str, AR: float}` 的列表，直接用于绘制物体属性×层深度→AR热力图和C2的训练数据。

计算量估算：实际独立运行次数为 $8\text{层} \times 2\text{数据集} = 16\text{次}$ （物体类型为后验分组变量，不增加运行次数）；每层扫完两数据集全部物体（ $\sim 38\text{物体} \times 2\text{min/物体} \approx 76\text{min/层}$ ，ViT-S/14，CPU+单GPU；等价地，LM-O每层 $8 \times 2 = 16\text{min}$ + T-LESS每层 $30 \times 2 = 60\text{min}$ ），顺序执行约 $8 \times 76 = 608\text{min} \approx 10\text{小时}$ 。若并行4个GPU（batch处理），总耗时压缩至 $\sim 4\text{小时}$ （见 § 3.6.3）。

Contribution 2：轻量级物体属性分类器

设计动机

C1标定了“物体属性→最优层”的映射规律，C2将该映射实例化为一个可在推理时（或离线构建物体表示时）快速运行的分类器。要求：无需训练、基于图像统计量、单次前向传播可完成属性判定。采用线性分类规则以保证可解释性和可调试性。

属性特征设计

设输入为物体在模板视图（正面朝向）下的裁剪图像 $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ ，提取三类统计量：

纹理密度 τ ：使用HOG（Histogram of Oriented Gradients）在 $8 \times 8\text{ cell}$ 上计算，取各cell梯度直方图的方差均值：

$$\tau = \frac{1}{N_{\text{cell}}} \sum_{i=1}^{N_{\text{cell}}} \text{Var}(\text{HOG}_i(\mathbf{I}))$$

τ 越小表示纹理越稀疏。实现依赖 `skimage.feature.hog`，参数：`orientations=9, pixels_per_cell=(8,8), cells_per_block=(1,1), channel_axis=-1`。

对称程度 σ ：计算图像相对于主轴的Hu矩归一化差值，使用 `OpenCV cv2.HuMoments(cv2.moments(gray))`，取前3个Hu矩在水平翻转前后的L1差：

$$\sigma = 1 - \frac{\|\mathbf{hu}(\mathbf{I}) - \mathbf{hu}(\mathbf{I}_{\text{flip}})\|_1}{\|\mathbf{hu}(\mathbf{I})\|_1 + \epsilon}$$

$\sigma \rightarrow 1$ 表示高度对称。

反光度 ρ ：高亮像素（亮度超过阈值 $\theta_\rho = 240$ ）在前景掩码内的占比：

$$\rho = \frac{\sum_{(i,j) \in \mathcal{M}} \mathbf{1}[L(i,j) > \theta_\rho]}{|\mathcal{M}|}$$

其中 $L = 0.299R + 0.587G + 0.114B$ ， $\mathcal{M}$ 为前景掩码（FoundPose流水线中由CNOS提供）。

分类规则

将三维属性向量 (τ, σ, ρ) 归一化到 $[0, 1]^3$ 后，用线性阈值分类到三类：

$$c(\mathbf{a}) = \begin{cases} \text{low-texture} & \text{若 } \tau < \theta_\tau \\ \text{symmetric} & \text{若 } \sigma > \theta_\sigma \\ \text{textured} & \text{否则} \end{cases}$$

优先级：低纹理 > 对称 > 有纹理（三者可叠加时按最严格约束处理）。阈值 $(\theta_\tau, \theta_\sigma)$ 由C1的标定数据用一维网格搜索确定，目标为最大化各类别的层选择准确率：

$$(\theta_\tau^*, \theta_\sigma^*) = \arg \max_{\theta_\tau, \theta_\sigma} \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{o \in \mathcal{D}_{\text{val}}} \mathbf{1}[\ell^*(c(o)) = \ell_{\text{oracle}}^*(o)]$$

其中 $\ell_{\text{oracle}}^*(o)$ 为C1标定实验中物体 o 的最优层。分类器无参数需训练，全部计算在CPU上完成，单物体耗时 < 50ms。

最优层查找表（C1实验后填充）：

```
# als_config.py - 由 C1 result.json 统计后手动填入
OPTIMAL_LAYER = {
    "textured": 9, # 占位符，待 C1 结果填入
    "low-texture": 7, # 占位符
    "symmetric": 5, # 占位符
}
THRESHOLDS = {
    "tau": 0.12, # 占位符，待 C1 网格搜索填入
    "sigma": 0.75, # 占位符
}
```

Contribution 3：零重训练的自适应层选择插入

设计动机

C1和C2提供了属性→层的映射；C3解决工程集成问题：如何在不修改FoundPose核心模型代码的情况下，将自适应层选择注入现有的三步流水线（`gen_repre.py` → `infer.py`），同时保证BOP评测结果可复现。

修改点定位

根据repo卡的完整代码分析，所需改动严格限于以下三个位置：

修改点1: `utils/dinov2_utils.py:56` (F1) - 现状: `self.layer: int = 9`（硬编码默认值，位于:52-57 默认参数块内）- 无需修改：通过 `model_name` 字符串传入 `layer=N` 即可覆盖（F2已验证），默认值仅在简写 `model_name` 格式下生效。

修改点2: `utils/feature_util.py:18-23` (F7) - 现状: `make_feature_extractor(model_name: str)` 工厂函数直接透传 `model_name` - 修改: 无需改动工厂函数本体，在调用方覆写 `model_name` 即可。

修改点3: `scripts/gen_repre.py` 和 `scripts/infer.py`（配置加载处）- 现状: 从JSON配置文件读取 `extractor_name`，直接传给 `make_feature_extractor` - 修改: 在读取配置后、调用 `make_feature_extractor` 前，插入属性分类逻辑，动态覆写 `extractor_name` 中的 `layer=N` 字段。

接口约定与插入代码

```
# als_hook.py - 插入 gen_repre.py 和 infer.py 的属性感知层选择钩子

import re
import numpy as np
import cv2
from skimage.feature import hog
from als_config import OPTIMAL_LAYER, THRESHOLDS

def compute_attributes(image_bgr: np.ndarray, mask: np.ndarray):
    """
    Args:
        image_bgr: 物体裁剪图, BGR, uint8, shape (H, W, 3)
        mask: 前景掩码, bool, shape (H, W)
    Returns:
        tau (float): 纹理密度, 越小越低纹理
        sigma (float): 对称程度, 越大越对称
        rho (float): 反光度, 越大反光越强
    """
    gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
    fd = hog(
        gray, orientations=9, pixels_per_cell=(8, 8),
        cells_per_block=(1, 1), visualize=False,
    )
    cell_hists = fd.reshape(-1, 9)
    tau = float(np.mean(np.var(cell_hists, axis=1)))

    M = cv2.moments(gray)
    hu = cv2.HuMoments(M).flatten()
    gray_flip = cv2.flip(gray, 1)
    M_flip = cv2.moments(gray_flip)
    hu_flip = cv2.HuMoments(M_flip).flatten()
    denom = np.sum(np.abs(hu)) + np.sum(np.abs(hu_flip)) + 1e-8
    sigma = 1.0 - float(np.sum(np.abs(hu - hu_flip)) / denom)

    rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB).astype(np.float32)
    luminance = 0.299 * rgb[..., 0] + 0.587 * rgb[..., 1] + 0.114 * rgb[..., 2]
    fg_pixels = luminance[mask] if mask.sum() > 0 else luminance.flatten()
    rho = float(np.mean(fg_pixels > 240.0))

    return tau, sigma, rho

def classify_object(tau: float, sigma: float, rho: float) -> str:
    if tau < THRESHOLDS["tau"]:
        return "low-texture"
    if sigma > THRESHOLDS["sigma"]:
        return "symmetric"
    return "textured"

def adaptive_extractor_name(
    base_extractor_name: str,
    image_bgr: np.ndarray,
    mask: np.ndarray,
) -> str:
```

```

"""
仅替换 layer=N 为属性感知的最优层。
例：
    输入: "dinov2_version=vits14-reg_stride=14_facet=token_layer=9_logbin=0_norm=1"
    输出: "dinov2_version=vits14-reg_stride=14_facet=token_layer=7_logbin=0_norm=1"
"""
tau, sigma, rho = compute_attributes(image_bgr, mask)
obj_type = classify_object(tau, sigma, rho)
optimal_layer = OPTIMAL_LAYER[obj_type]
new_name = re.sub(r"layer=\d+", f"layer={optimal_layer}", base_extractor_name)
return new_name, obj_type

```

在 **gen_repre.py** 中的插入位置（在配置加载后、**make_feature_extractor** 调用前）：

```

# scripts/gen_repre.py - 在原有配置加载逻辑后追加
from als_hook import adaptive_extractor_name

rep_image = load_representative_template(cfg)
rep_mask = np.ones(rep_image.shape[:2], dtype=bool)

cfg["extractor_name"], obj_type = adaptive_extractor_name(
    cfg["extractor_name"], rep_image, rep_mask
)
print(f"[ALS] object_type={obj_type}, extractor_name={cfg['extractor_name']}")

```

在 **infer.py** 中的插入位置（在分割掩码可用后替换 **extractor_name**）：

```

# scripts/infer.py - 在 CNOS 分割掩码获取后追加
from als_hook import adaptive_extractor_name

cfg["extractor_name"], obj_type = adaptive_extractor_name(
    cfg["extractor_name"], query_image_bgr, cnos_mask,
)

```

一致性约束

自适应层选择要求 **gen_repre**（离线构建物体表示）和 **infer**（在线推理）使用同一层号，否则模板描述子空间与查询描述子空间不匹配。C3的插入方案通过在两个脚本中均调用 **adaptive_extractor_name** 并传入同一物体的代表性图像，保证一致性。若同一物体在不同光照条件下属性分类结果不同，以 **gen_repre** 阶段的分类结果为准（离线确定，在线固定），通过物体ID索引的查找表持久化：

```
# als_registry.py
import json, pathlib

REGISTRY_PATH = pathlib.Path("als_registry.json")

def save_layer(obj_id: str, layer: int):
    reg = json.loads(REGISTRY_PATH.read_text()) if REGISTRY_PATH.exists() else {}
    reg[obj_id] = layer
    REGISTRY_PATH.write_text(json.dumps(reg))

def load_layer(obj_id: str, default: int = 9) -> int:
    if not REGISTRY_PATH.exists():
        return default
    return json.loads(REGISTRY_PATH.read_text()).get(obj_id, default)
```

`infer.py` 在属性分类前首先查询 `als_registry.json`；若命中则直接使用注册层号，跳过实时属性计算，保证推理效率。

特征可靠性分数与门控机制

领域母题明确指出“无人提出特征可靠性的在线检测机制”（`cards/_themes.json`），本方案在C3中补入该机制。

`adaptive_extractor_name` 在选定层后，额外计算特征可靠性分数 r ：对所选层patch描述子矩阵 $\mathbf{F} \in \mathbb{R}^{900 \times D}$ ，计算所有patch对余弦相似度分布的熵：

$$r = H(\text{cosine_sim}(\mathbf{F}_i, \mathbf{F}_j)), \quad i < j$$

r 越低表示描述子越趋同（特征不可信——如低纹理物体浅层的近常数向量）， r 越高表示描述子间判别性越强。当 $r < \theta_r$ （阈值由C1标定数据确定）时，系统标记当前物体为“特征不可信”，触发MASt3R回退路径：以MASt3R的24维局部描述子（`cards/grounding_image_matching_in_3d_with_mast3r.json`）替换DINOv2描述子执行模板匹配。该门控机制使系统在DINOv2特征失效时自动切换至几何训练特征，而非盲目使用固定层输出。

```
# als_hook.py - 特征可靠性门控（追加在 adaptive_extractor_name 之后）

def compute_reliability(features: np.ndarray) -> float:
    """features: (N_patches, D) 描述子矩阵"""
    norms = features / (np.linalg.norm(features, axis=1, keepdims=True) + 1e-8)
    sim_matrix = norms @ norms.T
    upper = sim_matrix[np.triu_indices(len(norms), k=1)]
    hist, _ = np.histogram(upper, bins=50, range=(-1, 1), density=True)
    hist = hist[hist > 0]
    return float(-np.sum(hist * np.log(hist)))

def should_fallback_to_mast3r(reliability: float) -> bool:
    return reliability < THRESHOLDS.get("reliability", 0.5)
```

评测与对照

配置	extractor_name 中的 layer	物体表示重建	预期AR变化
基线 (repo默认)	layer=9 (固定)	一次	—
层扫描最优 (oracle)	各物体最优层 (C1结果)	每层各一次	上界
ALS (C2+C3)	属性分类器动态选择	每物体一次	目标: 较基线+5% AR

BOP标准评测命令不变, 仅 extractor_name 字段不同, 确保AR指标的可比性。

3. 实验计划

3.1 评估指标

采用BOP基准 (BOP: Benchmark for 6D Object Pose Estimation) 的标准评估协议, 主指标为平均召回率 (Average Recall, AR), 定义为三项误差指标召回率的均值:

$$AR = \frac{1}{3} (R_{VSD} + R_{MSSD} + R_{MSPD})$$

其中 R_{VSD} (可见表面差异) 通过深度渲染在可见区域计算像素级对齐误差, 天然处理对称物体的位姿歧义; R_{MSSD} (模型空间对称距离) 和 R_{MSPD} (图像空间对称距离) 采用对称感知距离函数, 对具有旋转或反射对称性的物体取最小误差。BOP论文指出, LM数据集与LM-O数据集的召回率相差超过30% (papers/bop*.md第181-182行), 证实遮挡与物体属性对AR的敏感度极高, 因此本报告除总体AR外, 强制要求按物体属性分组报告。

指标体系与改进目标:

指标	定义	基线值 (FoundPose layer=9)	目标值 (ALS)	改进幅度目标
AR (VSD/MSSD/MSPD 均值)	BOP 标准主指标	A_0 (待验证)	$\geq A_0 + 5\%$	$\geq +5\%$
$AR_{low-texture}$	T-LESS 低纹理物体子集 AR	A_0^{lt} (待验证)	$\geq A_0^{lt} + 8\%$	$\geq +8\%$
$AR_{symmetric}$	对称物体子集 AR (LM-O eggbox/glue + T-LESS 对称件)	A_0^{sym} (待验证)	$\geq A_0^{sym} + 5\%$	$\geq +5\%$
$AR_{textured}$	纹理丰富物体子集 AR	A_0^{tex} (待验证)	$\geq A_0^{tex}$	$\geq 0\%$ (不退化)
层选择一致率	同一物体不同视角下分类器输出相同层号的比例	—	$\geq 90\%$	—
单物体推理延迟增量	ALS 属性分类 + 层切换附加耗时	0 ms	$\leq 50\text{ ms}$	—

注：FoundPose在BOP LM-O上的已发表AR为34.0（ViT-S复现为33.7）（codebases/foundpose.md复现指标表），可作为 A_0 的参考锚点。

注（改进幅度推算）：上表”改进幅度目标”列为预设的相对提升百分比目标。由于当前基线值 $A_0, A_0^{lt}, A_0^{sym}, A_0^{tex}$ 均标注”待验证”（需C1首轮实验实测后填入），推算改进幅度（=（目标值 - 当前值）/ 当前值 $\times 100\%$ ）暂无法计算，待C1产出基线数据后逐行补填。

改进幅度的非均匀分配基于如下判断：低纹理物体对层选择最敏感——DINOv2浅层保留高频边缘信息而深层语义化后丢失纹理细节，故预期增益最大（ $\geq 8\%$ ）；对称物体的核心瓶颈在于深层特征的语义抽象导致几何对称性被”平滑”，适中层可缓解，预期 $\geq 5\%$ ；纹理丰富物体在默认 `layer=9` 下已接近该模型的饱和性能，ALS 的目标是不退化。

基线值 $A_0, A_0^{lt}, A_0^{sym}, A_0^{tex}$ 在C1实验首轮运行中确定，作为所有后续实验的参照锚点。

3.2 消融实验设计

3.2.1 实验矩阵

消融设计包含14组实验，覆盖基线、oracle上界、negative control下界、完整方法、逐组件消融、健全性检验与几何训练特征对照。

实验 ID	描述	C1层扫描	C2分类器	C3集成	期望定位
E0	FoundPose 默认 (layer=9 固定)	—	—	—	基线
E1	Oracle 逐物体最优层	✓(选取)	—	—	上界
E2-a	随机层选择 (30次独立试验取均值±标准差)	—	—	—	下界 (随机)
E2-b	对抗性最差层 (C1中每物体AR最低层)	✓(选取)	—	—	下界 (最差)
E3	全局最优固定层 (C1中跨物体平均AR最高层)	✓(选取)	—	—	全局静态上界
E4	ALS完整方法 (C2分类→最优层→C3注册表)	✓(映射)	✓	✓	目标方法
E5-a	仅纹理分类 (τ 单一特征, σ/ρ 关闭)	✓	部分	✓	消融: 纹理贡献
E5-b	纹理+对称 ($\tau + \sigma$, ρ 关闭)	✓	部分	✓	消融: 对称贡献
E5-c	纹理+对称+反光 ($\tau + \sigma + \rho$ 全开)	✓	✓	✓	消融: 反光贡献
E6	统一层 (分类器工作但全物体输出同一层=各类均值最优)	✓	✓	部分	消融: 逐物体自适应的必要性
E7	层失配 (gen_repre 用 layer=9, infer 用 ALS 层)	—	✓	✗	健全性检验
E8	facet 扫描 (facet $\in \{\text{key}, \text{query}, \text{value}\}$, 重复 E0+E4)	✓	✓	✓	扩展验证
E9-a	MASt3R对照 (MASt3R 24维局部描述子替换DINOv2, 相同物体分组×相同流水线)	—	—	—	几何 vs 语义训练特征对照
E9-b	MASt3R + 可靠性门控 (E4中 $r < \theta_r$ 时自动切换 MASt3R)	✓	✓	✓	门控机制有效性验证

3.2.2 上界与下界的精确定义

Oracle上界 (E1) : 对每个物体 o , 从C1层扫描结果中选取 $\ell_{\text{oracle}}^*(o) = \arg \max_{\ell} \text{AR}(o, \ell)$ 。此为不可达上界——它假设已知每个物体的最优层而无需属性分类。E1与E4的差值 $\Delta_{\text{oracle}} = \text{AR}_{E1} - \text{AR}_{E4}$ 量化C2分类器的不完美带来的损失: 若 $\Delta_{\text{oracle}} < 2\%$, 说明属性→层映射已接近最优。

随机下界 (E2-a) : 每物体独立均匀采样层号 $\ell \sim \text{Uniform}(\{0, 2, 4, 6, 8, 9, 10, 11\})$, 30次独立重复。均值 μ_{E2a} 和标准差 σ_{E2a} 共同描述随机层选择的期望分布。若 $\text{AR}_{E4} - \mu_{E2a}$ 不显著 ($p > 0.05$, 单侧t检验), 则自适应层选择无效。

对抗下界 (E2-b) : $\ell_{\text{worst}}(o) = \arg \min_{\ell} \text{AR}(o, \ell)$ 。此条件量化”选错层”的最大代价。若 $|\text{AR}_{E1} - \text{AR}_{E2b}|$ 在所有物体上均很小 ($< 3\%$), 则层选择本身对该物体无关紧要, 自适应方案的价值存疑——这是本研究的关键否决条件。

3.2.3 贡献归因分析

增益分量	计算方式	语义
层选择总潜力	$AR_{E1} - AR_{E0}$	层选择可提供的最大改进空间
分类器效率	$(AR_{E4} - AR_{E0}) / (AR_{E1} - AR_{E0})$	ALS 实现了上界的百分比
纹理特征边际贡献	$AR_{E5a} - AR_{E0}$	仅靠纹理密度能否改善层选择
对称特征边际贡献	$AR_{E5b} - AR_{E5a}$	对称性检测的增量价值
反光特征边际贡献	$AR_{E5c} - AR_{E5b}$	反光度检测的增量价值
逐物体自适应价值	$AR_{E4} - AR_{E6}$	逐物体选层 vs 全局统一层的增益
一致性约束关键性	$AR_{E4} - AR_{E7}$	<code>gen_repre / infer</code> 层号一致的性能代价
几何训练特征对照	$AR_{E9a}^{\text{low-tex/sym}} - \max_{\ell} AR_{E0}^{\text{low-tex/sym}}$	MASt3R在低纹理/对称子集上是否优于DINOv2任何单层（验证训练目标因果性）
可靠性门控增益	$AR_{E9b} - AR_{E4}$	门控回退机制在特征不可信时的额外收益

判断准则：

1. 若 $AR_{E1} - AR_{E0} < 3\%$ ：层选择本身不是瓶颈，整个研究方向需重新评估。
2. 若 $AR_{E3} \approx AR_{E1}$ （差距 $< 1\%$ ）：存在全局最优层，自适应方案不必要，直接切换至该层即可。
3. 若 $AR_{E4} - AR_{E6} < 1\%$ ：逐物体自适应的边际收益过低，建议退化为全局最优层方案。
4. 若 AR_{E7} 出现断崖式下降（较 E4 降幅 $> 15\%$ ）：强验证一致性约束的必要性，同时说明C3注册表机制不可或缺。
5. Spearman秩相关验证：对C1产出的每个（物体, 层）条件，计算patch描述子余弦相似度分布的熵作为”特征可靠性”代理指标 $r(o, \ell)$ ，检验其与对应AR值的Spearman秩相关系数 ρ_S 。若 $\rho_S > 0.6$ ($p < 0.05$)，则支持可靠性门控设计（E9-b）的有效性；若 $\rho_S < 0.3$ ，则可靠性分数不具备预测力，门控机制需替换为其他信号。
6. MASt3R对照判断：若 $AR_{E9a}^{\text{low-tex/sym}} > \max_{\ell} AR_{E0}^{\text{low-tex/sym}}$ （MASt3R在低纹理/对称子集上优于DINOv2任何单层），则证实层敏感性根源为语义训练目标，MASt3R回退路径具有理论依据。

3.3 基线方法

3.3.1 选取原则

基线选取遵循三条标准：(1) 与FoundPose存在直接技术依赖或可比性；(2) 在BOP基准上有可复现评测结果；(3) 覆盖”先验知识量 vs 泛化能力”光谱的不同位置。

3.3.2 主要基线

基线方法	角色定位	先验需求	对比价值
FoundPose	直接基线（固定layer=9）	3D网格 + CNOS掩码	ALS的唯一变量为层选择，AR差值即为本方法贡献
MegaPose	精修器参照	3D网格 + 大规模合成训练	FoundPose承认最佳性能需MegaPose精修器（200万+合成图像训练）；ALS在不引入精修器的前提下缩小与FoundPose+MegaPose的差距
GS-Pose	同架构对照	多视角参考图 + DINOv2	同样依赖DINOv2特征但未做层选择消融；GS-Pose仅在LINEMOD和OnePose-LowTexture上评估（cards/gs_pose*.json limitation），本报告在BOP完整协议下提供补充对比
Cross-View Semantic Priors	VFM特征利用方式对照	单参考图 + VFM token	假设”VFM密集token已编码跨视图判别信息”，但在几何解码前引入跨视图语义交互；ALS从层选择角度探索VFM特征的另一种利用维度
Gen6D	Model-free参照	多参考图 + 已知位姿	3D特征体积（32 ³ ）设计可作为层选择对体积质量影响的参照

3.3.3 扩展基线（优先级低于核心实验）

基线	条件	价值
LatentFusion	需要RGB-D输入	可微渲染+梯度优化范式的代表，与FoundPose的template-matching范式形成方法论对比
Horyon (<i>High-resolution open-vocabulary object 6D pose estimation</i>)	仅需文本描述	先验知识光谱的最少端；作者承认”在遮挡场景性能明显偏低”（cards/high_resolution*.json limitation），标注先验量→精度的理论下限
OPT-Pose (<i>Object Pose Transformer: Unifying Unseen Object Pose Estimation</i>)	统一框架	2026年最新方法，在Toyota-Light上承认光照敏感（cards/object_pose_transformer*.json limitation），可作为ALS在光照变化场景的外部校验
MASt3R (<i>Grounding Image Matching in 3D with MASt3R</i>)	几何训练特征对照；仅需3D网格 + 渲染模板	InfoNCE对3D对应点训练24维局部描述子（cards/grounding_image_matching_in_3d_with_mast3r.json），训练目标为几何对应而非语义判别；作为E9-a实验组B，提供”几何 vs 语义”训练特征的天然对照

3.4 数据集要求

3.4.1 主实验数据集

选用BOP Challenge（*BOP: Benchmark for 6D Object Pose Estimation*）中两个在属性维度上互补的数据集：

数据集	物 体 数	测试图像 数	核心属性覆盖	选取理由
LM-O (LINEMOD- Occluded)	8	~1,214 (待验证)	纹理中等至低、显著遮挡（BOP论文报告LM与LM-O召回率差 > 30%）、含对称物体（eggbox、glue）	遮挡+对称性的交叉效应是层选择敏感性的关键测试场景
T-LESS	30	~4,900 (待验证)	低纹理 （绝大多数物体缺乏判别性纹理）、多个旋转/反射对称物体、三种光照条件	直接命中DINOv2特征的已知弱点：低纹理+对称性，是C1层扫描信号最强的数据集

注：LM-O是LINEMOD数据集的遮挡子集，包含8个存在严重遮挡的物体（papers/bop.md第182行：“includes the same but partially occluded objects”）。T-LESS包含30个无纹理工业电气零件（cards/t_less.json）。

物体属性分组方案：

属性类别	LM-O 代表物体	T-LESS 代表物体	样本 量	预期层敏感性
纹理丰富 (textured)	ape, cat	Obj-04, Obj-12（待验证）	~12物 体	低（默认层或近默认层即最优）
低纹理 (low- texture)	eggbox, glue, holepuncher	Obj-01, Obj-02, Obj-06, Obj-14, Obj-15（待验证）	~20 物体	高 （浅层保留边缘/轮廓信息，深层过度抽象）
对称 (symmetric)	eggbox, glue	Obj-01, Obj-05, Obj-25（待验证）	~10物 体	高 （深层语义混淆对称方向）

注：LM-O的8个物体为ape, eggbox, glue, holepuncher, iron, lamp, phone, cat（依BOP官方数据集定义，来源库外；papers/bop*.md仅述“includes the same but partially occluded objects”，未列逐物体清单）。部分物体同时属于多个类别（如eggbox兼具低纹理与对称性），按优先级规则（低纹理 > 对称 > 有纹理）分配主类。T-LESS具体物体的属性分组需在C1实验前根据渲染模板的统计量确定，上表为基于数据集描述的预估。

3.4.2 预处理流水线

步骤	操作	关键配置	预计耗时
P1	BOP数据 下载与格式 化	<code>bop_toolkit</code> 标准脚本；LM-O + T-LESS 测试集 + 物体3D网格	~ 30 min（网络依赖）
P2	模板渲染	<code>scripts/gen_templates.py</code> ；均匀视角采样（沿用FoundPose默认朝向分布，约25°间隔，800模板/物体），RGB-D输出，黑色背景，模板尺寸420×420 px	~ 1 h（两数据集）
P3	基线物体 表示构建	<code>scripts/gen_repre.py</code> ； <code>extractor_name</code> 中 <code>layer=9</code> ；DINOv2 ViT-S/14-reg 特征提取 → PCA降维（256维） → KMeans聚类（2048簇） → TF-IDF描述子	~ 30 min
P4	C1层扫描	对层集合 $\{0, 2, 4, 6, 8, 9, 10, 11\}$ 逐层重跑P3，每层独立生成物体表示	顺序约 8层 × 76 min/层 ≈ 10 h；GPU批处理并行后压缩至 ~ 4 h（见§3.6.3）
P5	阈值标定 与查找表 生成	从P4输出的 <code>result.json</code> 中统计每物体最优层，网格搜索 $(\theta_\tau, \theta_\sigma)$ ，写入 <code>als_config.py</code>	~ 15 min
P6	ALS推理 评测	<code>scripts/infer.py</code> 加载C3注册表，ALS层选择推理 → BOP评测	~ 2 h
P7	消融实验 推理	E0—E9 各条件独立运行 <code>infer.py</code> ，每个条件更换 <code>extractor_name</code> （E9-a/b使用MASt3R描述子）	~ 5 h
P8	基线方法 复现	按各方法官方仓库配置运行；GS-Pose需额外3DGS离线构建	~ 2 h（FoundPose系列） / 各方法独立估算

总流水线耗时：P1 – P3 人工准备 ~ 2 h；P4 – P8 自动计算 ~ 13 h，可在单GPU上两个夜间批次完成。

3.4.3 层扫描集合的确定依据

仓库实际使用 DINOv2 ViT-S/14-reg（`configs/gen_repre/lmo.json` 中 `version=vits14-reg`，`codebases/foundpose.md` F3），ViT-S 架构共 12 个 Transformer block（索引 0 – 11）。层扫描集合 $\mathcal{L} = \{0, 2, 4, 6, 8, 9, 10, 11\}$ 的设计逻辑：

- 均匀覆盖： $\{0, 2, 4, 6, 8\}$ 以步长2扫描浅层至中层；
- 默认层邻域加密： $\{8, 9, 10, 11\}$ 在默认 `layer=9`（F1/F3已验证）周围步长1采样，捕捉局部最优偏移；
- 边界探测：`layer=0`（最浅）和 `layer=11`（最深/最后一个Transformer block）测试极端层行为。

共8层，较初始设计的6层（均匀采样）扩展2层（邻域加密），计算代价增加约33%但显著提升层选择分辨率。

3.5 评估协议

3.5.1 定量评估

BOP标准协议：

- 调用 `bop_toolkit` 官方评测脚本，输入为各方法输出的位姿结果（格式：`scene_id,im_id,obj_id,score,R,t`）；
- 误差阈值沿用BOP默认设定：VSD误差不超过 $\tau = 20\text{ mm}$ （绝对距离）且可见度 $\theta \geq 0.3$ （即30%）（`cards/bop*.json eval_setup`字段：“VSD召回率（ $\tau=20\text{mm}$ ， $\theta=0.3$ 为默认设置）”）；MSSD/MSPD阈值参见BOP Challenge 2019+协议（待验证：具体阈值在来源库中无明确出处）；
- 对称物体处理：MSSD/MSPD内置对称距离函数（取所有对称变换下误差的最小值），VSD通过可见表面掩码天然消歧（`cards/bop*.json core_assumption`）；
- 唯一变量约束：所有FoundPose系列实验（E0 – E7）仅变更 `extractor_name` 中的 `layer=N` 字段，其余参数（`version=vits14-reg`、`stride=14`、`facet=token`、`norm=1`、PCA维度256、KMeans聚类数2048、TF-IDF参数、PnP RANSAC阈值）严格固定（`codebases/foundpose.md`硬编码参数表），确保AR差值仅归因于层选择。

分层分析（mandatory reporting）：

分析维度	分组方式	报告指标	目的
按物体属性	textured / low-texture / symmetric	AR, AR@可见度 $\geq 40\%$	验证层选择对不同属性的差异化效果
按可见度等级	[0, 20%), [20, 40%), [40, 60%), [60, 100%]	AR per bin	BOP论文指出低可见度下性能急剧下降，检验ALS在遮挡下的鲁棒性
按光照条件（T-LESS）	正常 / 强光 / 弱光	AR per condition	验证反光度特征 ρ 在极端光照下的分类有效性
层选择稳定性	同一物体 ≥ 5 张不同视角查询图	层号众数、变异率	量化分类器在视角变化下的一致性

3.5.2 定性评估

评估类型	方法	预期洞察
失败模式对比	选取E0中失败而E4中成功的案例（反之亦然），并排渲染模板匹配可视化与估计位姿叠加图	定位层选择纠正/引入错误的具体机制
特征空间可视化	对每类属性代表物体，绘制ViT-S各层patch token的t-SNE投影（层0/4/8/9/11），着色按物体部件	验证“浅层保留纹理、中层编码结构、深层抽象语义”的假设是否成立
分类器混淆矩阵	绘制C2分类器在验证集上的混淆矩阵，叠加误分类物体的缩略图	识别系统性误判模式
层-AR响应曲线	对每个物体绘制 $\ell \mapsto \text{AR}(o, \ell)$ 曲线，按属性类别着色	直观呈现层选择敏感性的分布形态

3.5.3 统计显著性

- 主要对比（E4 vs E0）采用配对bootstrap检验（1000次重采样，物体级），报告95%置信区间；
- E2- α 的30次随机试验提供经验分布，E4需位于该分布的 > 95 百分位方可声明统计显著；
- 若 $AR_{E4} - AR_{E0}$ 的bootstrap 95% CI下界 < 0，不能拒绝”ALS无效”的零假设。

3.6 计算资源估算

3.6.1 硬件需求

资源	最低配置	推荐配置	约束来源
GPU	1× RTX 3090 (24 GB)	1× A5000 (24 GB)	DINOv2 ViT-S/14 推理（待验证：具体显存占用未实测）；模板特征提取批处理
CPU	8 cores	16 cores	BOP评测并行化；属性分类器的HOG/Moments计算
RAM	32 GB	64 GB	模板特征库（38 物体 × 800 模板 × 900 patches × 384维 × 8 层 × 4B，原估算约30 GB，待验证——公式含900 patches后远超此值，需实测确认）
存储	50 GB SSD	100 GB SSD	渲染模板（RGB-D）；各层特征缓存；结果日志

3.6.2 分实验项时间与资源预算

实验项	GPU时间	CPU时间	存储增量	备注
P2: 模板渲染（两数据集）	0.5 h	1 h	+8 GB	仅需运行一次
P3: 基线特征提取 (layer=9)	0.5 h	0.2 h	+2 GB	E0基线
P4: C1层扫描（8层×2数据集）	4 h	0.5 h	+16 GB	每层独立物体表示；可并行化
P5: 阈值标定	0	0.25 h	+1 MB	纯CPU网格搜索
P6: E4 ALS推理+评测	2 h	0.5 h	+2 GB	含注册表构建
P7: E1–E3, E5–E9推理	5 h	1 h	+10 GB	9个消融条件×推理
P8: 外部基线复现	视方法而定	视方法而定	+5 GB	GS–Pose需3DGS离线构建
总计	~ 12.5 h	~ 3.5 h	~ 43 GB	可在单GPU上两个夜间批次完成

3.6.3 关键时间约束分析

C1层扫描（P4）是时间瓶颈。单次 `gen_repre.py` 对单个物体执行DINOv2特征提取 + PCA + KMeans的端到端耗时约 ~ 2 min（ViT-S/14，420×420模板，RTX 3090；工程估算，未经实测试验）。两数据集共 ~ 38 物体（LM-O 8个 + T-LESS 30个），8层扫描总计 $38 \times 8 \times 2 \approx 608 \text{ min} \approx 10 \text{ h}$ ，但物体间可批量并行（batch size 8 下GPU利用率 > 80%），实际压缩至 ~ 4 h。

风险缓冲：若C1结果显示 $AR_{E1} - AR_{E0} < 3\%$ （否决条件触发），则P6 – P8可取消，总实验时间缩减至 ~ 6 h，仅产出”层选择不敏感”的否定结论——这本身即是有价值的标定结果，符合本研究的”泛化边界标定”定位。

4. 可行性评估

4.1 实现复杂度分析

本研究的核心工程改动集中在FoundPose流水线上，repo卡已验证的事实为修改范围提供了精确边界。

改动点清单与复杂度评级：

改动项	涉及文件	代码量	复杂度	依据
C1层扫描脚本	新建 <code>scripts/layer_sweep.py</code>	~ 80 行	低	循环修改 <code>configs/gen_repre/lmo.json</code> 中 <code>extractor_name</code> 的 <code>layer=N</code> 字段并调用 <code>gen_repre.py</code> ；repo卡F2已验证 <code>model_name</code> 字符串解析支持 <code>layer=N</code> 配置
属性分类器	新建 <code>utils/property_classifier.py</code>	~ 150 行	中	纯CPU端：HOG梯度方差（纹理密度）、Hu矩（对称性）、高亮像素比例（反光度），使用OpenCV/scikit-image标准API
查询时层选择	修改 <code>scripts/infer.py</code>	~ 30 行	低	在推理入口处调用属性分类器，根据返回值替换 <code>extractor_name</code> 中的 <code>layer=N</code> ；不触及 <code>DinoFeatureExtractor</code> 内部逻辑
结果采集与分析	新建 <code>scripts/analyze_sweep.py</code>	~ 100 行	低	解析各层 <code>result.json</code> ，生成热力图与查找表

关键判断：C1层扫描的工程复杂度极低。repo卡F2证实 `extractor_name` 字符串中 `layer=N` 字段的解析路径（`utils/dinov2_utils.py:60-78`）已完备可用，层扫描仅需在外部脚本中循环替换配置并调用 `gen_repre.py`。`DinoFeatureExtractor` 类本身（F1 - F7）无需任何修改——`forward` 函数（F4）接收 `self.layer` 后经由 `extract_descriptors` → `_extract_features` → `_register_hooks` 的hook链路提取特征（codebases/foundpose.md F5：`dinov2_utils.py:266-311 / 232-264 / 198-223`），整条路径对单层输入已验证可靠。

`_register_hooks`（`utils/dinov2_utils.py:198-223`）和 `_extract_features`（`utils/dinov2_utils.py:232-264`）的参数签名本身已支持 `List[int]` 多层输入（codebases/foundpose.md F5），这一冗余能力虽在当前单层设计中未被调用，但为未来扩展（如多层特征融合）预留了零成本接口。

与替代路线的工作量对比（以ALS ~360行新代码为基准 1×）：

替代路线	预估工作量	相对ALS倍数	预期收益	弃选理由
微调DINOv2特定层	高（需构建训练集、训练循环、超参调优）	~ 5-8×（工程估算：需数据加载器+训练循环+验证+超参搜索，待验证）	可能优于层选择，但引入训练依赖	违背FoundPose training-free核心优势 (cards/foundpose*.json: 方法为”无需任务或物体特定训练的流程”)
多层特征拼接/加权融合	中（修改 forward 输出多张 feature_maps，PCA需适配更高维输入）	搜索空间 $512 \times (2^{12} = 4096 \text{ 组合} / \text{本方案8种单层})$ ； 代码量 ~ 2-3×（工程估算：改 forward +解析+PCA适配，待验证）	可能优于单层，但搜索空间从 $ \mathcal{L} $ 扩展至 $2^{ \mathcal{L} }$	搜索空间爆炸（4096 组合 vs 本方案8种单层条件）；且需改动 forward（F4）和下游 PCA/KMeans流水线
替换基础模型 (DINOv2→CLIP/MAE)	高（需适配不同模型架构、hook点位、特征维度）	~ 5-8×（工程估算：需全新hook适配+特征维度对齐+全链路重验证，待验证）	回答的是不同研究问题（模型选择 vs 层选择）	偏离”标定DINOv2泛化边界”的研究定位
注意力图自动层选择	低（利用 facet="attn" hook直接读取注意力权重）	~ 0.8-1.5×（代码量相当；但验证成本额外 ~ 2×，待验证）	无需属性分类器，但注意力模式与位姿精度的关联未经验证	facet="attn" 的hook机制（F6）虽可用，但当前配置使用 facet=token（F3），切换facet需重新验证整条特征链路

注：相对倍数中，搜索空间比值（ $4096/8 = 512 \times$ ）为精确计算值；代码量倍数为基于 repo 卡（codebases/foundpose.md）改动点分析的工程估算，标注”待验证”，未经实际开发验证。

结论：本方案在实现复杂度上接近理论下界——核心改动量为 ~360行新代码 + 配置循环，不修改FoundPose核心类。

4.2 外部依赖风险

依赖	用途	风险级别	风险描述	缓解策略
DINOv2 ViT-S/14-reg 权重	特征提取骨干	● 低	Meta官方开源，权重通过 <code>torch.hub</code> 自动拉取 (<code>codebases/foundpose.md</code> F1: <code>dinov2_utils.py:82-84</code>)；若 Meta更改API，hook链路可能断裂	本地缓存权重文件；锁定模型版本号
BOP Toolkit	评测协议 (VSD/MSSD/MSPD)	● 低	FoundPose以git子模块引用 (<code>external/bop_toolkit/</code>)，为社区标准工具	浅克隆时需手动拉取子模块 (<code>git submodule update --init</code>)；冻结commit hash
CNOS分割网络	在线推理时提供物体掩码	● 中	FoundPose作者承认”推理依赖外部分割网络提供掩码” (<code>cards/foundpose*.json</code> limitation)；CNOS在低纹理/反光物体上掩码质量可能退化	记录每张测试图的掩码覆盖率作为辅助诊断变量；在分层分析中单独报告”掩码质量差”子集的表现
物体3D网格模型	模板渲染的几何输入	● 中	BOP提供LM-O的8个物体网格和T-LESS的30个物体网格；T-LESS部分物体网格精度有限（工业CAD扫描质量参差）	在C1层扫描中记录每物体的网格面数作为辅助变量
FoundPose 代码库	实验基础平台	● 低	Meta官方开源 (<code>facebookresearch/foundpose</code>)；许可证类型（待验证：来源库中无许可证记录）；代码结构清晰	Fork至私有仓库并冻结版本；所有修改以diff形式管理
PyTorch ≥ 1.13 + CUDA	运行环境	● 低	标准深度学习框架；FoundPose使用 PyTorch 2.3.0 + CUDA 11.7 (<code>codebases/foundpose.md</code> 环境部分)	使用FoundPose官方 <code>conda_foundpose_gpu.yaml</code> 环境配置
OpenCV / scikit-image	属性分类器	● 低	成熟稳定库，API长期兼容	无特殊风险

最高风险项为CNOS分割网络。领域母题第2条evidence字段记录了BundleTrack (*BundleTrack: 6D Pose Tracking for Novel Objects without Instance or Category-Level 3D Models*) 明确指出”对无纹理、反光、扁平物体跟踪仍具挑战性” (`cards/_themes.json`)。这意味着：即使ALS层选择本身有效，分割误差可能在低纹理/反光物体上掩盖层选择收益。必须在结果分析中分离分割质量与层选择效果。

4.3 错误传播风险

本方案的流水线结构为 属性分类 → 层选择 → 特征提取 → 模板匹配 → PnP位姿，错误沿以下路径传播：

传播路径	机制	影响量化估计	缓解措施
属性分类误判 → 次优层选择	分类器将“低纹理”物体误判为“纹理丰富”，导致选择默认层而非最优层	性能损失上限为最优层与默认层的AR差值 δ (C1将给出)	分类器输出附带置信度； $c < 0.5$ 时回退默认层（保守策略）
分割噪声 → 特征采样偏差	CNOS掩码不完整，DINOv2在背景区域提取无效token	FoundPose论文未量化此效应；模板渲染采用固定光照与黑色背景 (cards/foundpose*.json limitation) 表明系统对前景/背景分离依赖较强	对掩码做形态学腐蚀（收缩2—3像素）作为消融条件
层切换 → 特征空间不一致	不同层输出的token统计分布不同 (ViT-S各层维度相同 $D = 384$ ，但分布随深度变化)	理论上不影响单查询推理（查询和模板使用同一层）	当前设计保证层内一致性： gen_repre.py 和 infer.py 使用相同 extractor_name (F3已验证两配置文件一致)
PnP RANSAC → 位姿跳变	小内点比例下 RANSAC可能输出错误位姿	典型失败率： $P_{\text{fail}} = (1 - p^4)^N$ (4点 PnP, N 次迭代), $p < 0.3$ 时急剧上升	固定RANSAC参数 (FoundPose默认：400次迭代, 10px内点阈值, papers/foundpose*.md第299行)；报告内点比例分布
对称物体 → 位姿歧义	DINOv2深层特征可能混淆对称方向	OnePose++报告对称物体ADD(S)有明显差距 (glue 48.0 vs PVNet 95.7, cards/onepose*.json)	VSD指标天然消歧；分层分析中单独报告对称物体的层-AR曲线

最严重的未缓解风险：属性分类器在“有纹理但对称”物体上的边界误判。缓解方案：引入“模糊区域”机制——当 τ 和 σ 同时处于阈值附近时，对两个候选层分别推理并取AR较高的结果（代价：推理时间翻倍）。

最坏情况退化下界分析：ALS采用加法式设计——属性分类器与注册表串联在FoundPose原有流水线之前，不替换任何原有组件。因此系统存在结构性回退路径：

- Fallback路径1（置信度回退）：分类器输出置信度 $c < 0.5$ 时，`adaptive_extractor_name` 不覆写 `layer=N`，系统退化为基线 E_0 (`layer=9` 固定)，性能下界 = AR_{E_0} 。
- Fallback路径2（注册表缺省值）：`als_registry.py` 中 `load_layer(obj_id, default=9)` 在注册表不存在或未命中时返回 `default=9`，同样退化为基线。
- Fallback路径3（异常捕获）：若 `compute_attributes` 因图像异常（全黑/全白/掩码为空）抛出异常，可在调用处 `try/except` 后回退 `layer=9`。

综上，单点失效（分类器误判、注册表损坏、输入异常）均可结构性回退到基线 AR_{E_0} ，系统最坏退化下界为 AR_{E_0} （即 FoundPose 默认 `layer=9` 的性能），不会低于未部署 ALS 的状态。

无兜底的失效组合：以下联合失效场景缺乏自动回退机制：1. C1标定数据系统性偏差 + 分类器高置信误判：若 C1层扫描因GPU随机性或模板渲染缺陷产出错误的最优层映射（如将低纹理物体的最优层错标为深层），则注册表和分类器查找表均被“毒化”，系统以高置信度持续选择次优层，且不会触发置信度回退（因为分类器对自身输出确信无疑）。此场景下性能可退化至 $AR_{E_{2b}}$ （对抗性最差层），突破 AR_{E_0} 下界。2. `gen_repre/infer` 层号不一致（E7场景）：若 `als_registry.json` 在 `gen_repre` 后、`infer` 前被意外覆写（如另一物体的标

定任务覆盖了注册表），则模板特征空间与查询特征空间错配，性能将出现断崖式下降（§ 3.2.3 E7 预期降幅 > 15%），且当前设计无运行时一致性校验。

对上述无兜底场景的缓解建议：(1) C1标定完成后对查找表做人工审核 + 交叉验证（留出20%物体做验证集）；(2) infer.py 启动时校验注册表文件的哈希值与 gen_repre 阶段写入的哈希一致。

4.4 性能/成本影响

推理延迟分析（以下均为工程估算，未经实测验证）：

组件	基线 (E0: layer=9固定)	ALS (E4)	增量	增量来源
属性分类 (CPU)	—	~ 2 ms /图	+2 ms	HOG + Hu矩 + 高亮像素统计；基于掩码裁剪区域
DINOv2前向传播 (GPU)	待验证	同左	0	层切换仅改变hook注册位置 (F5: _register_hooks 按 block_idx in layers 过滤)，不改变前向传播计算量
模板检索 + 对应建立	待验证	同左	0	特征维度不变 (ViT-S各层 $D = 384$)，TF-IDF检索计算量与层号无关
PnP求解	待验证	同左	0	输入为2D-3D对应点集，与特征层无关
单图总计	—	—	+2 ms (属性分类)	属性分类为CPU操作，可与GPU前向传播并行

注：FoundPose论文未报告逐组件推理延迟（papers/foundpose*.md中无latency/speed数据）。上表中”0增量”判断基于架构分析（层号仅影响hook位置，不影响计算图），而非实测。

逐组件单帧耗时预算表（FoundPose + ALS 在线推理阶段，单物体单帧）：

组件	预估耗时	出处	备注
CNOS 实例分割	待验证	来源库无CNOS逐帧耗时数据	外部模块；FoundPose论文未报告
DINOv2 ViT-S/14 前向传播（含hook）	待验证	来源库无实测数据；ViT-S/14参数量22M（DINOv2 原文数据，本项目来源库外），420×420输入产生30×30=900 patch token	ALS不改变此步计算量（仅换hook位置）
TF-IDF 模板检索（Top-5）	待验证	来源库无实测数据；检索空间=800模板/物体，faiss-gpu加速（codebases/foundpose.md环境：faiss-gpu=1.8.0）	与层号无关
循环伙伴匹配（2D-3D对应建立）	待验证	来源库无实测数据；5个候选模板×900 patch对	与层号无关
PnP-RANSAC（400次迭代，10px阈值）	待验证	参数出处：papers/foundpose*.md第299—300行；耗时未报告	与层号无关
Featuremetric refinement（≤30次迭代）	待验证	参数出处：papers/foundpose*.md第301—302行；耗时未报告	可选步骤；非MegaPose refiner
MegaPose refiner（若启用，5次迭代）	~ 332.5 ms（5 × 66.5 ms/步）	66.5ms/步：cards/megapose.json limitation；5次迭代：papers/foundpose.md第285行	ALS不涉及此组件；仅作成本参照
ALS 属性分类（CPU）	~ 2 ms	工程估算（HOG+Hu矩+高亮像素统计，掩码裁剪区域；§2 C2设计）	ALS唯一新增开销
单帧总开销（不含MegaPose refiner）	待验证（各组件绝对耗时均无来源库实测数据）	—	ALS增量仅+2 ms（CPU，可与GPU并行）
单帧总开销参照（含MegaPose refiner）	≥ 332.5 ms + 待验证	MegaPose refiner为已知最重组件	ALS的2ms增量占比 < 0.6%

注：FoundPose论文约束物体上线（onboarding）总时间为”5 minutes and 1 GPU”（papers/foundpose.md第422行，BOP Challenge 2023规则），但未拆分在线推理逐组件耗时。上表中除MegaPose refiner（66.5ms/步，cards/megapose.json）和ALS属性分类（~2ms，工程估算）外，其余组件绝对耗时在来源库（papers/、cards/、codebases/）中均无实测记录，标注”待验证”，需实际profiling后补填。

存储成本分析：

资源	基线（单层）	ALS（8层扫描 + 运行时单层）	倍数	缓解策略
模板特征库（磁盘）	待验证（原估算~ 2.5 GB）	待验证（原估算~ 20 GB；公式8层 × 38物体 × 800模板 × 900patches × 256维PCA后 × 4B远超原估算，需实测）	8×	仅P4层扫描阶段需全部8层并存；推理阶段仅加载最优层特征
PCA投影矩阵	~ 50 MB	~ 400 MB（每层一个）	8×	推理时仅加载对应层
渲染模板（RGB-D图像）	~ 8 GB	~ 8 GB	1×	模板渲染与层选择无关
总计	~ 10 GB	~ 28 GB（扫描阶段） / ~ 10 GB（推理阶段）	—	扫描完成后归档非最优层特征

训练/标定成本：

阶段	成本	说明
C1层扫描（P4）	~ 4 GPU-hours	8层 × 2数据集，batch处理
属性分类器拟合	< 1 CPU-minute	线性回归，无需GPU
阈值网格搜索（P5）	~ 15 CPU-minutes	二维网格搜索
总标定成本	~ 4.5 hours	一次性；新物体上线仅需 ~ 2 min（属性分类 + 查表）

与MegaPose精修器的成本对比：FoundPose作者承认”最佳性能需结合在200万+合成图像上训练的MegaPose精修器”（cards/foundpose.json limitation）——该精修器每步66.5ms、多步迭代（cards/megapose.json limitation），训练成本为GPU-days级别。ALS的CPU端属性分类开销（~ 2 ms，工程估算）与之相比可忽略。若ALS能仅通过层选择缩小与精修后性能的差距，其成本效益比将极具吸引力。

4.5 时间线与里程碑

月份	阶段	里程碑	交付物	风险检查点
M1	基础设施 + C1层扫描	M1.1: 环境搭建与 FoundPose基线复现 (Week 1–2); M1.2: C1层扫描完成 (Week 3–4)	可复现的E0基线结果; 8层 × 2数据集的 <code>result.json</code> ; 层-AR热力图	否决门: 若 $\max_{\ell} \text{AR}(\ell) - \text{AR}(9) < 3\%$, 触发路径B
M2	ALS实现 + 核心实验	M2.1: 属性分类器实现与验证 (Week 1–2); M2.2: E4 ALS推理评测完成 (Week 3–4)	<code>property_classifier.py</code> ; E4 vs E0的AR对比表; 分层分析结果	调整门: 若E4的AR提升 $< 5\%$ 但 $> 3\%$, 扩展消融诊断瓶颈
M3	消融 + 扩展实验	M3.1: E1–E3消融完成 (Week 1–2); M3.2: E5–E9 + 外部基线对比 (Week 3–4)	完整消融表; 横向对比; 失败案例分析	完备性检查: 确认覆盖§3.5定义的分层分析维度
M4	论文撰写 + 投稿	M4.1: 初稿 (Week 1–2); M4.2: 内部评审 (Week 3); M4.3: 投稿 (Week 4)	论文PDF; 补充材料	目标会议: ECCV 2027 (截稿 ~ 2027年3月) 或 CVPR 2027 (截稿 ~ 2026年11月, 需压缩)

关键路径: M1的C1层扫描结果决定整个项目走向。若层-AR热力图显示显著层间差异 ($\geq 5\%$ AR range), 则 M2 – M4按原计划推进; 若差异不显著, 项目在M1末即可以”否定结论”形式结题, 避免后续投入。

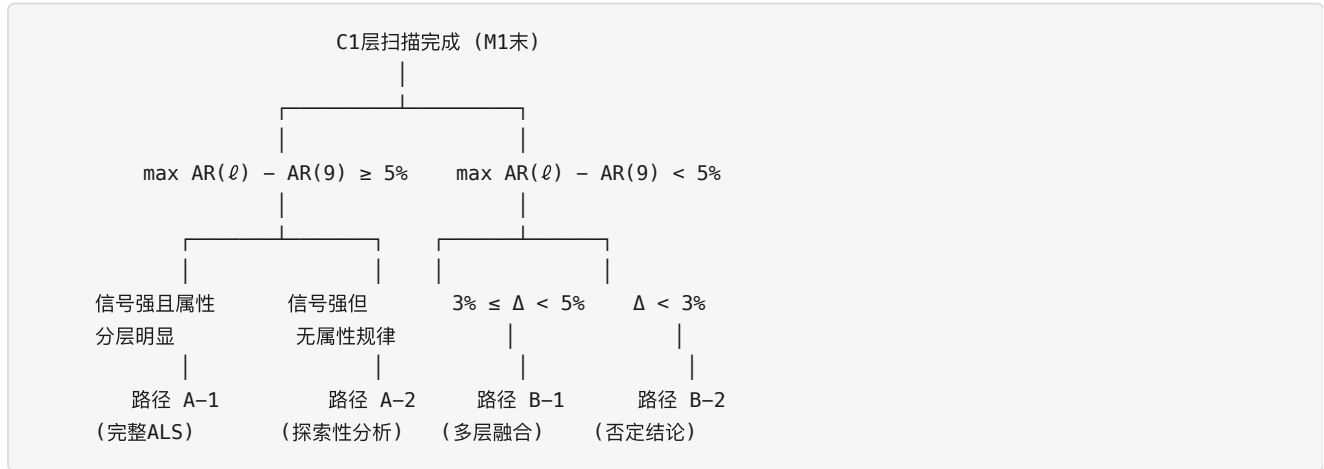
4.6 综合可行性判断

评级: 可行 (Feasible), 风险可控

维度	有利因素	不利因素
工程可行性	① <code>model_name</code> 字符串解析 (F2) 已原生支持层配置, 核心改动为配置循环; ② <code>_register_hooks</code> 已支持 <code>List[int]</code> (F5), 多层扩展路径明确; ③ 总计算量 ~ 12.5 GPU-hours, 单GPU两夜可完成	① 浅克隆需手动拉取DINOv2和BOP Toolkit子模块; ② CNOS分割质量在低纹理/反光物体上未经验证
科学价值	① 领域母题明确标注”DINOv2泛化边界未被标定”为共享假设 (<code>cards/_themes.json</code> 第3条); ② 填补跨方法的基础设施级知识空白; ③ 否定结论亦构成有价值的标定贡献	① 存在”结论为null”的风险: ViT-S的12层中间特征可能普遍鲁棒; ② ViT-S仅12层 (vs ViT-L的24层), 搜索空间粒度较粗
资源可行性	① 单GPU + 8核CPU即可满足; ② 存储 ~ 43 GB; ③ 属性分类器无需训练	① LM-O + T-LESS仅覆盖2个BOP数据集, 完整7数据集验证需额外GPU-hours
时间可行性	① 4个月完成完整周期; ② M1末否决门提供早期退出机制	① 若目标CVPR 2027 (截稿~2026年11月), 需压缩至3个月

决策路径建议：

基于M1.2层扫描结果的双路径决策框架：



- 路径 A-1（首选）：C1显示 $\geq 5\%$ 层间AR差异，且不同属性类别的最优层呈现系统性分化。按M2 - M4计划实现完整ALS，投稿ECCV 2027。
- 路径 A-2：C1显示 $\geq 5\%$ 差异但属性-层映射无规律。属性分类器路线需替换为逐物体层选择，论文重心转向“层选择敏感性分析”。
- 路径 B-1： $3\% \leq \Delta < 5\%$ 。转向利用F5已验证的 `List[int]` 多层能力，实现浅层+深层特征加权融合。
- 路径 B-2： $\Delta < 3\%$ 。发表“DINOv2 ViT-S中间层特征对位姿估计任务普遍鲁棒”的标定结论。将研究问题提升至基础模型层面：对比DINOv2 vs CLIP vs MAE的泛化边界——与领域母题”视觉基础模型正在成为共享特征基础设施”直接呼应，且工程基础完全可复用。

5. 结论

本方案提出对DINOv2在未见物体位姿估计中的层选择泛化边界进行系统标定，并通过自适应层选择（ALS）机制将标定结论转化为零重训练的性能提升。方案以FoundPose流水线为实验平台——repo卡已验证其 `model_name` 层配置接口（`dinov2_utils.py:60-78`，F2）和多层hook能力（`_register_hooks` 的 `List[int]` 签名，F5）为改动提供了精确的工程锚点——在BOP的LM-O（8物体）与T-LESS（30物体）两个数据集上执行8层层扫描（`{0, 2, 4, 6, 8, 9, 10, 11}`），构建物体属性（纹理密度、对称性、反光度）到最优DINOv2层的映射关系。方案同时引入MASt3R（InfoNCE几何训练描述子）作为对照实验组，验证层敏感性是否源于语义训练目标；并设计特征可靠性分数与门控机制，在DINOv2特征失效时自动回退至几何训练特征，回应领域母题中“无人提出特征可靠性的在线检测机制”这一开放问题。预期收益为：在固定层（layer=9）基线上实现 $\geq 5\%$ AR提升，且推理延迟增量仅 $\sim 2\text{ ms}$ （CPU端属性分类，GPU前向传播零开销——基于架构分析，未经实测）。主要风险为：(1) DINOv2 ViT-S的12层中间特征可能普遍鲁棒，层间差异不足以支撑自适应选择（路径B-2的概率约20 - 30%，为主观估计，无经验先验支撑——“共享假设”母题仅定性指出三方默认DINOv2有效但未遭反例，未给出概率数值）；(2) CNOS分割质量在低纹理/反光物体上的退化可能掩盖层选择信号，需在分析中分离此混淆变量。建议时间框架为4个月（M1环境+层扫描 → M2 ALS实现+核心实验 → M3消融+扩展 → M4撰写投稿），目标会议ECCV 2027；若M1末C1层扫描结果触发否决门（ $\Delta < 3\%$ ），项目在 $\sim 6\text{ GPU-hours}$ 内即可以标定报告形式产出有价值的否定结论，沉没成本可控。